



PROGRAM: *BSIT*

YEAR: 2

SEMESTER: 1

COURSE CODE: *CSC2105*

COURSE NAME: *Object Oriented Programming*

EXAMINATION DATE: 18th Sept 2026

DUE DATE: Friday, 18th Sept 2026 **4pm**

Instructions

Answer all questions by developing, running, testing and documenting Python programs on your computer. This assignment combines concepts already covered in class with selected concepts that students are expected to research independently and apply correctly.

The assignment assesses and extends your understanding of:

- classes and objects;
- `__init__()` and `self`;
- instance and class attributes;
- methods and object state;
- public, protected and private/name-mangled attributes;
- getters and setters;
- `@property` and validation;
- UML class diagrams;
- exception handling;
- file handling and persistence.

General requirements:

- Use meaningful class, variable and method names.
- Use correct indentation and useful comments.
- All submitted programs must execute successfully.
- Test both valid and invalid operations where the question requires them.
- Programs that only define classes without demonstrating them will not receive full marks.
- Where independent research is required, use credible programming resources and cite them.
- Every student must be prepared to demonstrate and explain any submitted code. Marks may be withheld where a student cannot reasonably explain code submitted under their name.

Question 1: UML Research and System Design [15 Marks]

Uganda Christian University wants to develop a small Smart Campus Services System. The proposed system should support students who use services such as printing, cafeteria purchases, laboratory services and selected departmental services. Each student has a campus account containing information such as student name, registration number, programme, account balance and daily spending limit. Transactions should also be recorded.

Before implementation begins, the development team requires a UML model.

a) UML Research [5 Marks]

Conduct independent research on UML Class Diagrams. Your research should clearly explain:

- what UML stands for;
- what a UML class diagram is used for;
- the three major sections of a UML class;
- how attributes are represented;
- how methods are represented;
- the meaning of the visibility symbols +, # and -;
- what an association between classes represents;
- what multiplicity means.

Include at least two credible references.

b) UML Class Diagram [7 Marks]

Design a UML class diagram for the Smart Campus Services System containing at least:

```
Student
StudentAccount
Transaction
CampusService
```

The diagram must show class names, meaningful attributes, meaningful methods, appropriate visibility symbols, relationships between classes, and appropriate multiplicities.

You may use draw.io, Lucidchart, Mermaid, or another suitable UML modelling tool.

c) Design Explanation [3 Marks]

Write a short explanation of your UML design. Explain why you selected the classes, which data should be publicly accessible, which data should be controlled, and how the classes interact.

Question 2: Classes, Objects and Encapsulation [20 Marks]

Using the design from Question 1, implement a simplified StudentAccount. Each account contains the student name, registration number, programme, balance, transaction PIN and daily spending limit. Not all information should have the same level of access.

a) Access-Level Design [4 Marks]

Use all three forms below and justify your choices in comments:

```
self.attribute
self._attribute
self.__attribute
```

Do not receive full credit merely by adding underscores randomly; your access-level choices must be sensible for the scenario.

b) Class Implementation [5 Marks]

Create a complete StudentAccount class using `__init__()`, `self`, meaningful instance attributes, at least one class attribute, and appropriate methods. Create at least three different StudentAccount objects.

c) Account Behaviour [5 Marks]

Implement methods that allow an account to deposit money, make a payment, and display an account summary. Demonstrate that changing one student's account does not automatically change another student's account.

d) Encapsulation Investigation [3 Marks]

Using comments and test statements, demonstrate and explain the difference between:

```
account.public_attribute
account._protected_attribute
account.__private_attribute
```

e) Name Mangling [3 Marks]

Research and explain name mangling. Demonstrate what Python does internally to a double-underscore attribute, and make it clear that `__attribute` is not encryption or absolute security.

Question 3: Getters, Setters, Validation and `@property` [20 Marks]

The first version of the StudentAccount system works, but management is concerned that sensitive account information may be modified incorrectly. Examples include a negative balance, a negative daily spending limit, or an empty programme name.

a) Classic Getter [3 Marks]

Create an appropriate getter method for one controlled attribute, such as `get_balance()`, and demonstrate how the getter is used.

b) Classic Setter [4 Marks]

Create a setter for another attribute. The setter must include meaningful validation. An invalid update must not destroy the last valid value.

c) Research `@property` [4 Marks]

Research Python's `@property` and explain what it does, why Python programmers may use it instead of explicit getter methods, what a property getter is, and what a property setter is.

d) Implement a Property [5 Marks]

Refactor one suitable attribute to use property syntax. For example:

```
@property
def daily_limit(self):
    ...

@daily_limit.setter
def daily_limit(self, value):
    ...
```

The setter must perform meaningful validation. Demonstrate reading the property, assigning a valid value, attempting an invalid value, and confirming that the object remains in a valid state.

e) Read-Only Property [4 Marks]

Add one attribute that should be readable but not directly assignable. Possible examples include account status, total number of transactions, or available daily balance. Implement it using a property getter without a corresponding property setter and demonstrate its use.

Question 4: Exception Handling and Program Reliability [20 Marks]

The Smart Campus Services System must now deal with incorrect user input and invalid transactions without unexpectedly terminating. Examples include negative deposits, zero payments, payments greater than the available balance, non-numeric input and invalid spending limits.

a) Exception Handling Research [5 Marks]

Research and clearly explain the purpose of:

```
try
except
```

```
else
finally
```

Also explain ValueError and give a simple example for each explanation.

b) Improve the Account Program [6 Marks]

Modify the StudentAccount program so that user-entered monetary values are handled safely. For example, entering 'five thousand' instead of 5000 should not cause the whole application to terminate unexpectedly.

c) Raise Your Own Exception [4 Marks]

Use raise ValueError(...) for at least one meaningful business rule. Example:

```
if amount <= 0:
    raise ValueError("Deposit amount must be greater than zero.")
```

The rule should be relevant to your system, not included merely to satisfy the requirement.

d) Test Cases [5 Marks]

Demonstrate at least five test cases:

- one valid deposit;
- one invalid deposit;
- one valid payment;
- one payment greater than the available balance;
- one non-numeric input.

For each test, clearly show the expected behaviour of the program.

Question 5: File Handling and Transaction Persistence [15 Marks]

At the moment, transactions disappear when the Python program ends. Management wants transaction records to remain available after the program has been closed. You are therefore required to introduce file handling.

a) File Handling Research [4 Marks]

Research and explain:

```
open()
read()
write()
close()
```

Also explain the difference between the file modes r, w and a, and state one appropriate situation for each mode.

b) Save Transactions [4 Marks]

Modify the application so that successful transactions are stored in a file. Each saved transaction should contain enough meaningful information to identify what happened, for example:

```
Registration Number
Transaction Type
Amount
Service
Remaining Balance
```

A possible stored record might look like:

```
S21B13/001 | PAYMENT | 5000 | CAFETERIA | 25000
```

c) Read Previous Transactions [3 Marks]

Create functionality that reads and displays previously stored transaction records. The program should be able to read information created during an earlier run.

d) Handle Missing Files [2 Marks]

Use exception handling to prevent the application from crashing if the expected transaction file does not yet exist.

e) Demonstration [2 Marks]

Demonstrate clearly that a transaction is written to the file, the program can finish, and the stored information can later be retrieved.

Question 6: Integrated Practical Challenge [10 Marks]

The Department of Computing and Technology wants you to demonstrate that your Smart Campus Services System now works as one coherent application. Using the work developed in Questions 1–5, produce a final integrated version.

a) Multiple Objects [2 Marks]

Create at least three different student accounts with different data.

b) Independent Object State [1 Marks]

Demonstrate that transactions performed on one account do not incorrectly modify another account.

c) Encapsulation [1 Marks]

Show meaningful use of public, `_protected` and `__private` attributes.

d) Controlled Access [2 Marks]

Demonstrate both a classic getter/setter and `@property` appropriately.

e) Exception Handling [1 Marks]

Demonstrate that at least one invalid operation is caught and handled.

f) File Persistence [1 Marks]

Demonstrate that at least one transaction remains available after the program ends.

g) Final Demonstration [2 Marks]

Run a short realistic scenario that demonstrates multiple users, valid and invalid transactions, saved transaction history and final account information.

A suitable final demonstration could include:

- Student A deposits money.
- Student A pays for printing.
- Student B pays for a cafeteria service.
- Student C attempts an invalid transaction.
- Transaction history is saved.
- Previous transactions are displayed.
- Final account information is shown.

TOTAL: 100 MARKS

Required Submission Structure

Submit one folder using the following structure:

```
CSC2105_Assignment1_RegistrationNumber/
|
|— README.md
|— Question1_UML/
|   |— UML_Research.pdf
|   |— Smart_Campus_UML.png
|— Question2_Encapsulation/
|   |— student_account.py
|— Question3_Properties/
|   |— controlled_account.py
|— Question4_Exceptions/
|   |— reliable_account.py
|— Question5_FileHandling/
|   |— transactions.py
```

```
| └─ transactions.txt
| └─ Question6_FinalSystem/
| └─ main.py
```

README Requirements

Student name and registration number.
Brief description of the system.
How to run the programs.
Concepts demonstrated.
Sources used for UML, @property, exception handling and file handling.
Any assumptions made.

Assessment Note

Because several parts of this assignment involve independent research, every student must be prepared to demonstrate and explain any part of the submitted code. Marks may be withheld where a student cannot reasonably explain the purpose or operation of code submitted under their name.

END